

SmartHealth AI Assistant Master Prompt & Jira Execution Guide

This PDF contains a complete professional execution blueprint for the Software Project Management final project: **SmartHealth AI Assistant – An Intelligent Healthcare Symptom Analysis & Appointment Management System**. The document includes: Master AI Prompt for generating the full project Complete Jira workflow implementation Agile Scrum execution plan Sprint breakdown Task assignment strategy Risk and quality management Documentation strategy Presentation & demo guidance

MASTER AI PROMPT

You are a senior full-stack software architect, AI engineer, DevOps engineer, QA lead, UI/UX designer, Scrum Master

Your task is to build a COMPLETE production-quality Final Year / Semester Project called:

SMARTHEALTH AI ASSISTANT

An Intelligent Healthcare Symptom Analysis & Appointment Management System

PROJECT OVERVIEW

- AI-powered symptom analysis using NLP
- Disease prediction using machine learning
- Specialist recommendation system
- Appointment booking and scheduling
- Patient dashboard and medical history
- Doctor management portal
- Admin panel
- Notifications and reminders
- JWT authentication and RBAC
- Complete Software Project Management workflow using Jira

TECH STACK

Frontend:

- React 18
- TypeScript
- Tailwind CSS
- Redux Toolkit
- Axios
- React Router
- Material UI
- Chart.js

Backend:

- Node.js
- Express.js
- Sequelize ORM
- PostgreSQL
- JWT Authentication
- Swagger/OpenAPI
- Socket.io

AI/ML Microservice:

- Python 3.11
- FastAPI
- spaCy
- NLTK
- scikit-learn
- TensorFlow
- Random Forest
- Transformers/BERT

DevOps:

- Docker
- Docker Compose
- GitHub Actions
- CI/CD
- Railway/Render/Vercel deployment

TESTING:

- Jest
- React Testing Library
- Pytest
- Cypress
- Supertest

GENERATE ALL MODULES:

1. Authentication Module
2. Patient Module
3. Doctor Module
4. Admin Module
5. AI Symptom Analysis Module
6. Disease Prediction Module
7. Appointment System
8. Notification System
9. Health Dashboard

SOFTWARE ENGINEERING REQUIREMENTS:

- Clean Architecture
- MVC pattern
- SOLID principles
- RESTful APIs
- Scalable folder structure
- Environment variables
- Error handling middleware
- Logging system
- Validation and sanitization
- Security best practices
- Rate limiting
- Helmet and CORS
- Swagger API docs
- Dockerization
- CI/CD workflows
- Unit tests and integration tests

DATABASE DESIGN:

Generate:

- ERD
- SQL schema
- Sequelize models
- Relationships
- Migration scripts
- Seeders

TABLES:

- Users
- Patients
- Doctors
- Appointments
- Symptoms
- DiseasePredictions
- Notifications
- MedicalRecords
- Reviews
- AuditLogs

AI/ML REQUIREMENTS:

- Symptom dataset preprocessing
- NLP pipeline
- Intent classification
- Symptom extraction
- Disease prediction model
- Model training pipeline
- Evaluation metrics
- Accuracy reports
- FastAPI microservice
- API integration with Node backend

UI/UX REQUIREMENTS:

- Modern responsive healthcare UI
- Mobile-first design
- Dark/light mode
- Dashboard layouts
- Form validation UX
- Loading states
- Toast notifications
- Reusable components

TESTING REQUIREMENTS:

- Unit tests
- Integration tests
- API tests
- End-to-end tests
- Security testing checklist
- Performance testing plan
- Coverage reports

DOCUMENTATION:

Generate:

- SRS
- SDD
- API Documentation
- Deployment Guide
- User Manual
- Testing Documentation
- Risk Management Plan
- Quality Management Plan
- Sprint Reports
- Final Project Report
- Presentation Slides
- Demo Script
- Jira Workflow Explanation
- Gantt Chart
- WBS

SPM REQUIREMENTS:

The project MUST fully satisfy:

- Jira project setup and workflows
- Sprint planning
- Agile Scrum execution
- Task decomposition
- Product backlog
- User stories
- Risk management
- Quality management
- Resource allocation
- Timeline management
- Progress tracking
- Kanban/Scrum boards
- Burndown charts
- Release management
- Team collaboration logs

OUTPUT FORMAT:

Provide:

- Folder structure first
- Step-by-step implementation
- Full code files
- File names before code
- Terminal commands
- Database setup
- Docker commands
- Deployment steps
- Jira setup instructions
- Testing procedures
- Professional documentation

Build the entire project incrementally and professionally as if it were being developed by a real software company

Jira Project Setup

Project Type	Scrum Project
Project Name	SmartHealth AI Assistant
Project Key	SHAI

Workflow Statuses

1	Backlog
2	Selected for Development
3	In Progress
4	Code Review
5	Testing
6	Blocked
7	Ready for Deployment
8	Done

Core Epics

Epic 1	Project Setup
Epic 2	Authentication System
Epic 3	Frontend UI
Epic 4	Backend APIs
Epic 5	Database System
Epic 6	AI Symptom Analysis
Epic 7	Disease Prediction
Epic 8	Appointment Management
Epic 9	Dashboard & Analytics
Epic 10	Testing & QA
Epic 11	Deployment & DevOps
Epic 12	Documentation

8 Sprint Plan

Sprint 1: Initiation & Setup

GitHub repo, Jira setup, architecture, ERD, frontend/backend setup, Docker.

Sprint 2: Authentication System

JWT auth, login, registration, RBAC, protected routes.

Sprint 3: Database & APIs

Sequelize models, PostgreSQL schema, CRUD APIs, Swagger docs.

Sprint 4: AI Symptom Analysis

NLP preprocessing, symptom extraction, FastAPI integration.

Sprint 5: Disease Prediction Engine

ML training, prediction APIs, recommendation logic.

Sprint 6: Appointment & Notifications

Booking system, reminders, calendar integration.

Sprint 7: Dashboard & Testing

Charts, analytics, Cypress tests, performance & security testing.

Sprint 8: Deployment & Documentation

Docker deployment, CI/CD, final report, PPT, demo recording.

Risk Management

Risk	Impact	Mitigation
AI accuracy low	High	Use multiple ML algorithms
Scope creep	High	Strict sprint goals
Deployment failure	Medium	Early Docker testing
Integration bugs	Medium	Early API testing
Security vulnerabilities	Critical	OWASP checks

Final Submission Checklist

- Working deployed application
- Frontend + Backend + AI Service
- PostgreSQL database
- Jira Scrum board
- Epics, stories, sprints, burndown charts
- Final report
- SRS and SDD
- QA and Risk reports
- PowerPoint presentation
- Demo video
- GitHub repository with commit history
- CI/CD setup